IT CAREER HUB

Django

Agile projects: Part 1



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

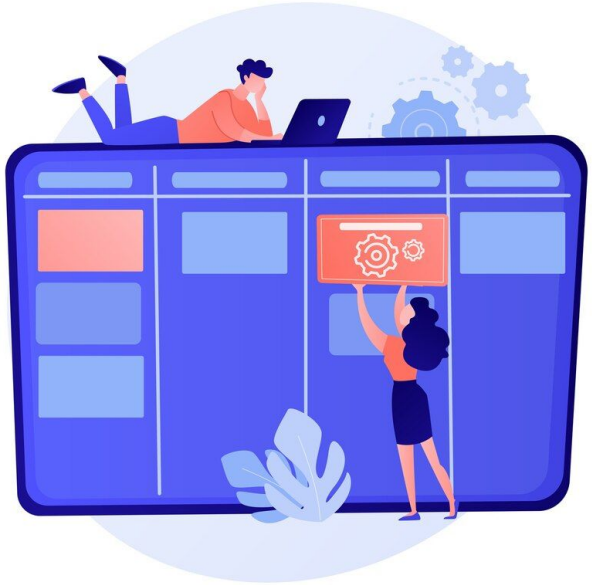
Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- Камера должна быть включена на протяжении всего занятия
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Вести себя уважительно и этично по отношению к остальным участникам занятия
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



Концепция проекта

Вы - Python разработчик в крупной компании и получили новый проект от клиента - "Agile Projects". Этот проект - будущая веб-платформа для управления проектами, задачами и сотрудниками, ориентированной на использование в рамках Agile методологии.



Цели проекта

Создать гибкую и удобную платформу для управления проектами, задачами и сотрудниками, с возможностью отслеживания статусов задач, их приоритетности и исполнителей.

Научиться выстраивать чёткую и понятную структуру проекта и кода, развить навыки выстраивания архитектуры проекта.



Основные требования

Платформа должна обеспечивать легкость в добавлении файлов к проектам и предоставлять пользователям инструменты для эффективного управления проектами в рамках Agile методологии, а также отслеживания аналитики проектов по количеству приходящих задач, скорости их закрытия и нагруженности проекта.

Проект должен реализовываться на фреймворке Django и придерживаться принципов REST для обеспечения удобного и стандартизированного взаимодействия с API.

IT CAREER HUB

УЧЕБНЫЙ ПРОЕКТ

Задание 1

Создание приложений, регистрация первой модели Tag

1. Для большего контроля и чистоты архитектуры создайте модуль apps, переместите только что созданное приложение в этот модуль.
2. Создайте первое приложение tasks.
3. Зарегистрируйте новое приложение в настройках Django.
4. Создайте модуль models в приложении tasks и создайте первую модель Tag в файле tag.py:
    - **name** (строковое поле, максимальная длина 20, минимальная длина - 4)
    - Строковое отображение объекта должно быть в виде его имени (магический метод **__str__**)
5. Проведите миграции, убедиться, что всё работает, запустив сервер.
6. Зафиксируйте все изменения, сделать запрос на слияние.

Задание 1

Решение

1. В пустом проекте создайте папку **apps** с пустым файлом `**__init__.py**` - модуль.
2. Перейдите в эту папку в консоли и введите команду **python ../manage.py startapp tasks.**
3. В настройках django найдите список **INSTALLED_APPS** и добавьте новую строку ‘apps.tasks.apps.TasksConfig’.
4. В новом приложении создайте модуль **models**:
    - Удалите файл **models.py.**
    - Создайте файл **tag.py** и в нём модель **Tag.**

Задание 1

Решение

```
from django.db import models
from django.core.validators import MinLengthValidator


class Tag(models.Model):
   name = models.CharField(max_length=20, validators=[MinLengthValidator(4)])

   def __str__(self):
       return self.name
```

Задание 1

Решение

5. В консоль введите команду ‘**python manage.py makemigrations**’ и следом за ней ‘**python manage.py migrate**’.

6. Git команды (**git add, git commit, git push origin <branch_name>**).

Задание 2

Второе приложение и модель Project

1. Создайте приложение **projects.**
2. Зарегистрируйте новое приложение в настройках Django.
3. Создайте модуль **models** в приложении **projects** и создайте первую модель **Project** в файле **project.py**:
    - **name** (строковое поле, максимальная длина 100, уникальное).
    - **description** (большое строковое поле, обязательно к заполнению).
    - **created_at** (Поле даты и времени. Заполняется автоматически при создании проекта).
    - Строковое отображение объекта должно быть в виде его имени (магический метод **__str__**).
    - Определение класса Meta для сортировки проектов по их имени.
4. Проведите миграции, убедиться, что всё работает, запустив сервер.
5. Зафиксируйте все изменения, сделать запрос на слияние.

Задание 2

Решение

1. Перейти в консоли в папку apps, введите команду `**python manage.py startapp projects**`.
2. В настройках django найти список **INSTALLED_APPS** и добавьте новую строку ‘apps.projects.apps.ProjectsConfig‘.
3. В приложении **projects** создайте модуль **models**:
    - файл **models.py** удалить.
    - Создайте файл **project.py** и в нём модель **Project.**

Задание 2

Решение

```
class Project(models.Model):
    name = models.CharField(unique=True, max_length=100)
    description = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name

    class Meta:
        ordering = ['-name']
```

Задание 2

Решение

4. В консоль введите команду ‘**python manage.py makemigrations**’ и следом за ней ‘**python manage.py migrate**’.

5. Git команды (**git add, git commit, git push origin <branch_name>**).

Задание 3

Модель файлов, построение связи с проектами

1. В приложении project создайте новую модель **ProjectFile**:
    - **file_name** (Строковое поле, максимальная длина - 120 символов)
    - **file_path** (Поле загрузки файлов, таргет сохранения - папка ‘documents’)
    - **created_at** (Дата создания файла, автозаполняется при создании нового файла)
    - Отображение объектов по полу name (магический метод **__str__**)
    - Сортировка записей по дате создания в порядке убывания
2. В модели **Project** создайте новое поле **files**, которое будет связывать проекты и файлы связью “Многие ко Многим”, связующее поле - “**project**”.
3. В модели Project добавьте динамическое поле **count_of_files** через декоратор **@property,** которое будет высчитывать количество файлов для проекта.
4. Проведите миграции, убедиться, что всё работает, запустив сервер.
5. Зафиксируйте все изменения, сделать запрос на слияние.

Задание 3

Решение

```
class Project(models.Model):
    …
    files = models.ManyToManyField('ProjectFile', related_name='project')

    @property
    def count_of_files(self):
       return self.files.count()



class ProjectFile(models.Model):
   file_name = models.CharField(max_length=120)
   file_path = models.FileField(upload_to='documents/')
   created_at = models.DateTimeField(auto_now_add=True)


   def __str__(self):
       return self.file_name

   class Meta:
       ordering = ['-created_at']
```

Задание 3

Решение

2. В консоль введите команду ‘**python manage.py makemigrations**’ и следом за ней ‘**python manage.py migrate**’.

3. Используйте Git команды (**git add, git commit, git push origin <branch_name>**).

Задание 4

Модель Задач, связь с проектами и пользователями. Часть 1

1. В приложении **tasks** создайте новую модель **Task**:
   - **name** (строковое поле, максимальное значение - 120 символов)
   - **description** (большое строковое поле, обязательно к заполнению)
   - **status** (строковое поле, максимальное значение - 15 символов, выбор из готовых полей через **Enum** класс **Statuses**, значение по умолчанию - **NEW**)
   - **priority** (Цифровое маленькое поле, возможность выбора из готовых значений через **Enum** класс **Priority**, значение по умолчанию - **MEDIUM**)
   - **project** (связующее **ForeignKey** поле к модели **Project**. При удалении проекта все задачи должны удаляться, поле для проектов - ‘tasks’)
   - **tags** (связующее **ManyToMany** поле к модели **Tag**, поле для тегов - ‘tasks’)
   - **created_at** (поле даты и времени. Автоматическое заполнение при создании записи)
   - **updated_at** (поле даты и времени. Автоматическое заполнение при создании и обновлении записи)
   - **deleted_at** (поле даты и времени, может быть пустым)
   - **deadline** (поле даты и времени, обязательно к заполнению. По умолчанию должен быть расчёт последнего дня текущего месяца при создании записи)
   - **assignee** (связующее **ForeignKey** поле к модели **User** от django. Защита от удаления, поле для пользователя - ‘tasks’, у задачи может не быть назначенного пользователя)

Задание 4

Модель Задач, связь с проектами и пользователями. Часть 2

2. Добавьте строковое отображение задач в формате: “название задачи, статус”.
3. Добавьте сортировку по дедлайн дате в порядке убывания.
4. Добавьте уникальность по полям name и project.
5. Проведите миграции, убедиться, что всё работает, запустив сервер.
6. Зафиксируйте все изменения, сделать запрос на слияние.

Задание 4

Решение

1. Создайте модель:

```
from django.contrib.auth.models import User
from django.db import models

from apps.projects.models.project import Project
from apps.tasks.choices.statuses import Statuses
from apps.tasks.choices.priority import Priority
from apps.tasks.utils.set_end_of_month import calculate_end_of_month
```

1

```
class Task(models.Model):
    name = models.CharField(
        max_length=120
    )
    description = models.TextField()
    status = models.CharField(
        max_length=15,
        choices=Statuses.choices,
        default=Statuses.NEW
    )
    priority = models.SmallIntegerField(
        choices=Priority,
        default=Priority.MEDIUM[0]
    )
    project = models.ForeignKey(
        Project,
        on_delete=models.CASCADE,
        related_name='tasks'
    )
```

2

Задание 4

Решение

```
tags = models.ManyToManyField('Tag', related_name='tasks')
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)
deleted_at = models.DateTimeField(null=True, blank=True)
deadline = models.DateTimeField(default=calculate_end_of_month)
assignee = models.ForeignKey(
    User,
    on_delete=models.PROTECT,
    related_name='tasks',
    null=True,
    blank=True
)

class Meta:
    ordering = ['-deadline']
    unique_together = ('name', 'project')

def __str__(self):
    return f"{self.name}, status: {self.status}"
```

Задание 4

Решение

2. Создайте функцию на расчёт последнего дня в месяце (apps/tasks/utils/set_end_of_month.py):

```
from datetime import datetime
import calendar

from django.utils import timezone


def calculate_end_of_month() -> datetime:
   current_date = timezone.now()
   amount_of_days = calendar.monthrange(
       current_date.year,
       current_date.month)[1]
   date = datetime(
       year=current_date.year,
       month=current_date.month,
       day=amount_of_days,
   )
   return date.astimezone()
```

Задание 4

Решение

3. Создайте Enum классы для статусов (apps/tasks/choices/statuses.py)**:**

```
from enum import Enum


class Statuses(Enum):
    NEW = "NEW"
    IN_PROGRESS = "IN_PROGRESS"
    PENDING = "PENDING"
    BLOCKED = "BLOCKED"
    TESTING = "TESTING"
    CLOSED = "CLOSED"

    @classmethod
    def choices(cls):
        return [(attr.name, attr.value) for attr in cls]
```

Задание 4

Решение

4. Создайте Enum классы для приоритетов (apps/tasks/choices/priority.py):

```
from enum import Enum


class Priority(Enum):
    VERY_LOW = (1, 'Very Low')
    LOW = (2, 'Low')
    MEDIUM = (3, 'Medium')
    HIGH = (4, 'High')
    CRITICAL = (5, 'Critical')

    @classmethod
    def choices(cls):
        return [(key.value[0], key.value[1]) for key in cls]

    def __getitem__(self, item):
        return self.value[item]
```

Задание 4

Решение

5. В консоль введите команду **‘python manage.py makemigrations’** и следом за ней **‘python manage.py migrate’.**

6. Используйте Git команды (**git add, git commit, git push origin <branch_name>**).

Задание 5

Получение списка всех тегов и создание нового тега

1. В приложении **tasks** удалите файл **views.py,** создайте модуль views.
2. В модуле views создайте новый файл **tag_views.py.**
3. В приложении **tasks** создайте новый модуль serializers.
4. В модуле **serializers** создайте новый файл **tag_serializers.py.**
5. Напишите сериализатор для работы с тегами (так как у нас всего одно поле в этой модели, сериализатор будет один на все будущие запросы).
6. Напишите классовое отображение:
    - Реализовать метод для получения всех объектов **Tag**
    - для получения списка всех тегов методом **get**
    - Для создания нового тега методом **post**
7. Зарегистрируйте новое классовое отображение в списке эндпоинтов.
8. Проверьте работу нашего классового отображения.
9. Зафиксируйте все изменения, сделать запрос на слияние.

Задание 5

Решение

1. Создайте сериализатор (apps/tasks/serializers/tag_serializers.py):

```
from rest_framework import serializers
from apps.tasks.models import Tag


class TagSerializer(serializers.ModelSerializer):

    class Meta:
        model = Tag
        fields = '__all__'
```

Задание 5

Решение

2. Создайте классовое отображение (apps/tasks/views/tag_views.py)**:**

```python
from rest_framework.request import Request
from rest_framework.response import Response
from rest_framework import status
from rest_framework.views import APIView

from apps.tasks.models import Tag
from apps.tasks.serializers.tag_serializers
      import TagSerializer
```

1

```python
class TagListAPIView(APIView):

    def get_objects(self) -> Tag:
        return Tag.objects.all()

    def get(self, request: Request) -> Response:
    tags = self.get_objects()

    if not tags.exists():
        return Response(
            data=[],
            status=status.HTTP_204_NO_CONTENT
        )
```

2

Задание 5

Решение

3. Регистрация классового отображения:

```python
# agile_projects/urls.py

from django.contrib import admin
from django.urls import path, include  # NEW

urlpatterns = [
   path('admin/', admin.site.urls),
   path('api/v1/', include('apps.routers')),  # NEW
```

1

```python
# apps/routers.py

from django.urls import path, include


urlpatterns = [
   path('tasks/', include('apps.tasks.urls')),
]
```

2

Задание 5

Решение

```
# apps/tasks/urls.py

from django.urls import path

from apps.tasks.views.tag_views import TagListAPIView


urlpatterns = [
   path('tags/', TagListAPIView.as_view()),
```

Задание 5

Решение

4. Запустите сервер, проверьте **GET** и **POST** запросы.

5. Используйте Git команды (**git add, git commit, git push origin <branch_name>**).

IT CAREER HUB

ВОПРОСЫ

Задание 6

Получение, обновление и удаление тега

1. В файле **tag_views.py** создайте новый класс **TagDetailApiView.**
2. Напишите отдельный метод, который позволит получать конкретный объект по его **id.**
3. Реализуйте метод **get** для получения конкретного тега.
4. Реализуйте метод **put** для возможности обновления информации о теге (обновление поля **name**).
5. Реализуйте метод **delete** для возможности удалить конкретный тег по его **id**
6. Проверьте работу нашего классового отображения.
7. Зафиксируйте все изменения, сделать запрос на слияние.

Задание 6

Решение

1. Создайте классовое отображение (apps/tasks/views/tag_views.py)**:**

```python
class TagDetailAPIView(APIView):
    def get_object(self, pk: int) -> Tag:
        return get_object_or_404(Tag, pk=pk)

    def get(self, request: Request, pk: int) -> Response:
        tag = self.get_object(pk=pk)

        serializer = TagSerializer(tag)

        return Response(
            serializer.data,
            status=status.HTTP_200_OK,
        )
```

1

```python
def put(self, request: Request, pk: int) -> Response:
    tag = self.get_object(pk=pk)

    serializer = TagSerializer(tag, data=request.data)

    if serializer.is_valid(raise_exception=True):
        serializer.save()

        return Response(
            serializer.validated_data,
            status=status.HTTP_200_OK,
        )

    return Response(
        serializer.errors,
        status=status.HTTP_400_BAD_REQUEST
    )
```

2

Задание 6

Решение

```
def delete(self, request: Request, pk: int) -> Response:
    tag = self.get_object(pk=pk)

    tag.delete()

    return Response(
        data={"message": "Tag was deleted successfully"},
        status=status.HTTP_200_OK
    )
```

Задание 6

Решение

2. Зарегистрируйте новое отображение в списке эндпоинтов (apps/tasks/urls.py)**:**

```
from apps.tasks.views.tag_views import TagListAPIView, TagDetailAPIView


urlpatterns = [
   path('tags/', TagListAPIView.as_view()),
   path('tags/<int:pk>/', TagDetailAPIView.as_view()),
]
```

Задание 6

Решение

3. Запустите сервер, проверьте **GET, PUT** и **DELETE** запросы.

4. Используйте Git команды (**git add, git commit, git push origin <branch_name>**).

Задание 7

Получение списка проектов и создание нового проекта. Часть 1

1. В приложении **projects** создайте модули **serializers**, **views**, удалить файл **views.py.**
2. В модуле **serializers** создайте файл **project_serializers.py.**
3. В только что созданном файле создайте первый сериализатор **AllProjectsSerializer** на получения информации о всех проектах. Отображаемые поля:
    - **id**
    - **name**
    - **Created_at**

Задание 7

Получение списка проектов и создание нового проекта. Часть 2

4. В модуле **views** создайте файл **project_views.py**, напишите классовое отображение:
    - Реализуйте метод на получение списка всех проектов:
        1. Получить из **query_params** значения дат **date_from**, **date_to**
        2. Если никаких значений передано не было - получить список полностью всех проектов
        3. Если были переданы значения для временного диапазона - сконвертировать полученные значения по текущей временной зоне (поможет метод **make_aware**() и модуль **timezone** в django)
        4. Провести фильтрацию по диапазону дат **date_from date_to**
    - Реализуйте метод **get** на отображение списка всех проектов, добавьте фильтрацию через **request_params** на получение проектов в определённом временном промежутке, например с **2020-01-01** по **2024-01-01**
    - Реализуйте метод на создание нового проекта

Задание 7

Получение списка проектов и создание нового проекта. Часть 2

5. Зарегистрируйте новое отображение в файле **urls.py** в приложении **projects.**
6. Проверьте работу нашего классового отображения.
7. Зафиксируйте все изменения, сделать запрос на слияние.

Задание 7

Решение

1. Создайте сериализаторы (apps/projects.serializers/project_serializers.py)**:**

```python
from rest_framework import serializers

from apps.projects.models import Project


class AllProjectsSerializer(serializers.ModelSerializer):

    class Meta:
        model = Project
        fields = ('id', 'name', 'created_at')
```

1

```python
class CreateProjectSerializer(serializers.ModelSerializer):
    created_at = serializers.DateTimeField(read_only=True)

    class Meta:
        model = Project
        fields = ('name', 'description', 'created_at')

    def validate_description(self, value: str) -> str:
        if len(value) < 30:
            raise serializers.ValidationError(
                "Description must be at least 30 characters long"
            )

        return value
```

2

Задание 7

Решение

2. Создайте классовое отображение (apps/projects/views/project_views.py)**:**

```python
class ProjectsListAPIView(APIView):
    def get_objects(self, date_from=None, date_to=None):
        if date_from and date_to:
            date_from = timezone.make_aware(
                datetime.strptime(date_from, '%Y-%m-%d')
            )
            date_to = timezone.make_aware(
                datetime.strptime(date_to, '%Y-%m-%d')
            ) if date_from else timezone.now().strftime('%Y-%m-%d')


            projects = Project.objects.filter(
                created_at__range=[date_from, date_to]
            )

            return projects

        return Project.objects.all()
```

1

```python
def get(self, request: Request) -> Response:
    date_from = request.query_params.get('date_from')
    date_to = request.query_params.get('date_to')

    projects = self.get_objects(date_from, date_to)

    if not projects.exists():
        return Response(
            data=[],
            status=status.HTTP_204_NO_CONTENT
        )

    serializer = AllProjectsSerializer(projects, many=True)

    return Response(
        serializer.data,
        status=status.HTTP_200_OK,
    )
```

2

Задание 7

Решение

```
def post(self, request: Request) -> Response:
    serializer = CreateProjectSerializer(data=request.data)

    if serializer.is_valid(raise_exception=True):
        serializer.save()

        return Response(
            serializer.validated_data,
            status=status.HTTP_201_CREATED,
        )

    return Response(
        serializer.errors,
        status=status.HTTP_400_BAD_REQUEST,
    )
```

Задание 7

Решение

3. Зарегистрируйте новый эндпоинт(apps/router.py):

```
from django.urls import path, include


urlpatterns = [
   path('tasks/', include('apps.tasks.urls')),
   path('projects/', include('apps.projects.urls')),  # NEW
]
```

(apps/projects/urls.py):

```
from django.urls import path

from apps.projects.views.project_views import
ProjectsListAPIView


urlpatterns = [
   path('', ProjectsListAPIView.as_view()),
]
```

Задание 7

Решение

4. Запустите сервер, проверьте **GET, POST** запросы.

5. Используйте Git команды (**git add, git commit, git push origin <branch_name>**).

Задание 8

Получение, обновление и удаление проекта

1. Создайте новый сериализатор **ProjectDetailSerializer** для получения конкретной информации по проекту:
    - **name**
    - **description**
    - **created_at**
    - **count_of_files**
2. Напишите новый класс-отображение **ProjectDetailAPIView** для получения конкретного проекта, обновления и удаления проекта.
    - При обновлении полученного проекта добавьте возможность частичного обновления полей.
3. Зарегистрируйте новый эндпоинт, проверьте как отрабатывают запросы **GET**, **PUT**, **DELETE.**
4. Зафиксируйте все изменения, сделайте запрос на слияние.

Задание 8

Решение

1. Создайте новый сериализатор (apps/projects.serializers/project_serializers.py):

```
class ProjectDetailSerializer(serializers.ModelSerializer):

    class Meta:
        model = Project
        fields = ('id', 'name', 'description', 'count_of_files')
```

Задание 8

Решение

2. Создайте класс-отображение ProjectDetailAPIView (apps/projects/views/project_views.py)**:**

```python
class ProjectDetailAPIView(APIView):
    def get_object(self, pk: int):
        return get_object_or_404(Project, pk=pk)

    def get(self, request: Request, pk: int) -> Response:
        project = self.get_object(pk=pk)

        serializer = ProjectDetailSerializer(project)


        return Response(
            serializer.data,
            status=status.HTTP_200_OK,
        )
```

1

```python
def put(self, request: Request, pk: int) -> Response:
    project = self.get_object(pk=pk)

    serializer = CreateProjectSerializer(
        instance=project,
        data=request.data,
        partial=True
    )
    if serializer.is_valid(raise_exception=True):
        serializer.save()

        return Response(
            serializer.validated_data,
            status=status.HTTP_200_OK,
        )
    return Response(
        serializer.errors,
        status=status.HTTP_400_BAD_REQUEST,
    )
```

2

Задание 8

Решение

```
def delete(self, request: Request, pk: int) -> Response:
    project = self.get_object(pk=pk)

    project.delete()

    return Response(
        data={
            "message": "Project was deleted successfully"
        },
        status=status.HTTP_200_OK,
    )
```

Задание 8

Решение

3. Зарегистрируйте новый эндпоинт (apps/project/urls.py):

```
urlpatterns = [
   path('', ProjectsListAPIView.as_view()),
   path('<int:pk>/', ProjectDetailAPIView.as_view()), # NEW
]
```

Задание 8

Решение

4. Запустите сервер, проверьте **GET, PUT** и **DELETE** запросы.

5. Используйте Git команды (**git add, git commit, git push origin <branch_name>**).

Задание 9

Работа с файлами. Часть 1

1. В приложении projects создайте новый модуль **utils.**
2. Создайте в этом модуле файл **upload_file_helpers.py.**
3. В этом файле необходимо создать функции:
    - Функция на проверку валидных расширений файла
    - Функция на проверку размерности файла (файл не должен быть больше 2 MB)
    - Функция на создание пути для сохранения файла
    - Функция на сохранение файла (файл должен записываться по частям для избежания потенциальных ошибок и потери данных)
4. В модуле **serializers** в приложении **projects** создайте новый файл **project_file_serializers.py.**
5. Создайте сериализатор **AllProjectFilesSerializer** для отображения краткой информации по всем файлам:
    - **id**
    - **file_name**
    - **project**

Задание 9

Работа с файлами. Часть 2

6. Создайте сериализатор **CreateProjectFileSerializer** для создания нового файла:
    - Для создания необходимо только поле **file_name**
    - Поле **file_name** необходимо валидировать:
        1. Проверка на наличие символов из таблицы **ASCII.**
        2. Проверка на доступные расширения файлов (**pdf, csv, doc, xlsx**).

7. Переопределите метод **create**:
    - Перед сохранением файла необходимо создать путь для сохранения
    - Так же необходимо получить из тела запроса сам файл и проверить его на размерность (**не более 2 MB**)
    - Если файл проходит все проверки - сохранить его в проекте, сохранить объект файла
    - Если файл слишком большой - поднять исключение валидации с сообщением об ошибке

Задание 9

Работа с файлами. Часть 3

8. Напишите класс-отображение **ProjectFileListAPIView** для получения списка всех файлов:
   - Реализовать метод на получение всех файлов для конкретного проекта по имени проекта. Если имя проекта не было передано - выводить список полностью всех файлов.

9. Добавьте в класс-отображение **ProjectFileListAPIView** метод post для создания нового файла и привязки его к конкретному проекту (проект получать через поле **id**, которое будет передаваться в теле запроса).

10. Зарегистрируйте новый эндпоинт, проверьте как отрабатывают запросы **GET**, **POST.**

11. Зафиксируйте все изменения, сделайте запрос на слияние.

Задание 9

Решение

1. Создайте файл upload_file_helpers.py с необходимыми функциями (apps/projects/utils/upload_file_helpers.py):

```python
import os
from pathlib import Path

ALLOWED_EXTENSIONS = ['.csv', '.doc', '.pdf', '.xlsx', '.py']

def check_extension(filename):
    extension = Path(filename).suffix

    if extension not in ALLOWED_EXTENSIONS:
        return False

    return True

def check_file_size(file, required_size=2):
    file_size = file.size / (1024 * 1024)

    if file_size > required_size:
        return False
    return True
```

1

```python
def create_file_path(file_name):
    new_file_name, file_ext = file_name.split('.')


    file_path = "documents/{}.{}".format(
        new_file_name,
        file_ext
    )

    return file_path

def save_file(file_path, file_content):
    os.makedirs(os.path.dirname('documents/'), exist_ok=True)

    with open(file_path, 'wb') as f:
        for chunk in file_content.chunks():
            f.write(chunk)

    return file_path
```

2

Задание 9

Решение

2. Создайте новые сериализаторы (apps/projects/serializers/project_file_serializers.py)**:**

```python
from rest_framework import serializers

from apps.projects.models import ProjectFile
from apps.projects.utils.upload_file_helper import (
    check_extension,
    create_file_path,
    check_file_size,
    save_file
)


class AllProjectFilesSerializer(serializers.ModelSerializer):

    class Meta:
        model = ProjectFile
        fields = ('id', 'file_name', 'project')
```

1

```python
class CreateProjectFileSerializer(serializers.ModelSerializer):
    class Meta:
        model = ProjectFile
        fields = ('file_name',)

    def validate_file_name(self, value: str) -> str:
        if not value.isascii():
            raise serializers.ValidationError(
                "Please, provide a valid file name."
            )
        if not check_extension(value):
            raise serializers.ValidationError(
                "Valid file extensions: ['.csv', '.doc', '.pdf',
'.xlsx']"
            )

        return value
```

2

Задание 9

Решение

```
def create(self, validated_data):
    file_path = create_file_path(
        file_name=validated_data['file_name']
    )
    raw_file = self.context.get('raw_file')

    if check_file_size(file=raw_file):
        save_file(file_path=file_path, file_content=raw_file)

        validated_data['file_path'] = file_path

        return ProjectFile.objects.create(**validated_data)

    else:
        raise serializers.ValidationError(
            "File size is too large (2 MB as maximum)."
        )
```

Задание 9

Решение

3. Создайте новый класс-отображение (apps/projects/views/project_file_views.py)**:**

```python
from rest_framework.request import Request
from rest_framework.response import Response
from rest_framework import status
from rest_framework.views import APIView
from rest_framework.generics import get_object_or_404

from apps.projects.models import ProjectFile, Project
from apps.projects.serializers.project_file_serializers import (
   AllProjectFilesSerializer,
   CreateProjectFileSerializer,
)
```

1

```python
class ProjectFileListAPIView(APIView):
   def get_objects(self, project_name=None):
       if project_name:
           project_file = ProjectFile.objects.filter(
               project__name=project_name
           )
           return project_file

       return ProjectFile.objects.all()
```

2

Задание 9

Решение

```
    def get(self, request: Request) -> Response:
        project_name = request.query_params.get('project')

        project_files = self.get_objects(project_name)

        if not project_files.exists():
            return Response(
                data=[],
                status=status.HTTP_204_NO_CONTENT
            )

        serializer = AllProjectFilesSerializer(project_files,
many=True)

        return Response(
            serializer.data,
            status=status.HTTP_200_OK
        )
```

3

```
def post(self, request: Request) -> Response:
    file_content = request.FILES["file"]
    project_id = request.data["project_id"]

    project = get_object_or_404(Project, pk=project_id)

    serializer = CreateProjectFileSerializer(
        data=request.data,
        context={
            "raw_file": file_content,
        }
    )
```

4

Задание 9

Решение

```
if serializer.is_valid(raise_exception=True):
    project_file = serializer.save()
    project_file.project.set([project])

    return Response(
        data={
            "message": "File upload successfully"
        },
        status=status.HTTP_200_OK
    )

else:
    return Response(
        data=serializer.errors,
        status=status.HTTP_400_BAD_REQUEST
    )
```

Задание 9

Решение

4. Зарегистрируйте новый эндпоинт (apps/projects/urls.py):

```
from apps.projects.views.project_file_views import (
   ProjectFileListAPIView,
)

urlpatterns = [
   path('', ProjectsListAPIView.as_view()),
   path('<int:pk>/', ProjectDetailAPIView.as_view()),
   path('files/', ProjectFileListAPIView.as_view()),
]
```

Задание 9

Решение

5. Запустите сервер, проверьте **GET, POST** запросы.

6. Используйте Git команды (**git add, git commit, git push origin <branch_name>**).

IT CAREER HUB

ВОПРОСЫ

Заключение

